
UD1. Introducción al uso de Python y su entorno

— Introducción al uso de Python —

¿En qué se utiliza?

Python es un lenguaje multipropósito empleado en una amplia variedad de aplicaciones, entre las que destacamos:

- **Ciencia y análisis de datos**
- **Inteligencia artificial**
- **Visualización de datos**
- **Desarrollo web backend**
- **Automatización y scripting**
- **IoT (internet de las cosas)**

Características del lenguaje

- **Lenguaje de alto nivel**: abstrae las instrucciones de forma que sean legibles para los humanos.
- **Interpretado**: el código fuente es leído, traducido y ejecutado en tiempo real por un programa llamado intérprete.
- **Multiparadigma**: admite diferentes paradigmas de programación, como POO, programación funcional y programación estructurada.
- **Multiplataforma**: se puede ejecutar en diferentes plataformas, como Windows, Linux y Mac OS.

Características del lenguaje

- **Tipado dinámico**: El tipo de la variable se decide en tiempo de ejecución y puede variar a lo largo de la misma.
- **Fuertemente tipado**: No podemos combinar variables de distinto tipo en la misma operación.

Entorno que vamos a usar

- Intérprete de Python:
 - <https://www.python.org/downloads/> (versión 3.14)
- IDEs:
 - <https://code.visualstudio.com/> (necesita la extensión para Python)
 - <https://www.jetbrains.com/es-es/pycharm>

Actividad

Instalar el intérprete de Python y los IDEs Visual Studio Code y PyCharm, y configurarlos para programar en Python en caso de ser necesario.

Sintaxis

Lo primero que tenemos que destacar de Python es que **no son necesarios los punto y coma** al final de cada sentencia (aunque no da error ponerlos).

Sintaxis

```
print('¡Hola mundo!')
```

```
# Esto es un comentario
```

```
''' y esto
```

```
también lo es
```

```
'''
```

Sintaxis

Como hemos dicho antes, las variables en Python son de tipado dinámico, lo que implica que no hay que poner su tipo al declararlas.

```
a = 1          # variable int
```

```
b = 3.5        # variable float
```

```
c = "Hola"     # variable string (es indiferente usar comillas  
simples o dobles)
```

```
d = True       # variable bool
```

Sintaxis

Podemos conocer el tipo de una variable con la función type.

```
a = 3
```

```
print(type(a))
```

Sintaxis

Podemos incluir saltos de línea en un print mediante el carácter de escape `\n`

```
print("Hola don Pepito\nHola don José")
```

Sintaxis

Podemos cambiar el valor de una variable en tiempo de ejecución, independientemente de su tipo.

```
a = 67
```

```
print(a)
```

```
print(type(a))
```

```
a = "League of Legends"
```

```
print(a)
```

```
print(type(a))
```

Sintaxis

También podemos asignar múltiples variables en una sola línea de código, así como imprimir varias variables en una única sentencia print.

```
a, b, c = 1, 2, 3
```

```
print(a, b, c)
```

Sintaxis

Para leer información de pantalla, usamos la función `input`, que puede incluir o no un texto que se muestre por pantalla.

```
a = input()
```

```
print(a)
```

```
nombre = input("Introduce tu nombre: ")
```

```
print(nombre)
```

El gestor de paquetes Pip

Pip es el gestor de paquetes de Python. Es un acrónimo recursivo de “Pip instala paquetes”.

Los paquetes permiten añadir nuevas funciones y herramientas a Python sin tener que programar el código desde cero.

Por ejemplo, el paquete Django ofrece numerosas herramientas a la hora de crear aplicaciones web.

El gestor de paquetes Pip. Comandos

- Instalar un paquete: `pip install nombre_del_paquete`
- Desinstalar un paquete: `pip uninstall nombre_del_paquete`
- Ver paquetes instalados: `pip list`
- Actualizar un paquete: `pip install --upgrade nombre_del_paquete`
- Guardar dependencias: `pip freeze > requirements.txt` (crea un archivo con la lista de librerías usadas en el proyecto)

Entornos virtuales

Un entorno virtual es un espacio aislado donde podemos instalar paquetes sin la necesidad de tener que instalarlos en la raíz del sistema.

Esto nos permite usar diferentes dependencias o incluso versiones de Python en diferentes proyectos sin causar conflictos.

Para crear un entorno virtual, usamos el siguiente comando:

```
python -m venv nombre-entorno
```

Entornos virtuales

A la hora de crear un entorno virtual, si tenemos varias versiones de Python instaladas en nuestro equipo, podemos elegir la que deseamos:

```
py -3.13 -m venv nombre-entorno
```

Entornos virtuales

Para poder instalar paquetes en el entorno virtual, tenemos que activarlo.

nombre-entorno\Scripts\activate

Y para desactivarlo usamos el siguiente comando

deactivate

Después de activar un entorno es recomendable asegurarnos de que está actualizado el gestor de paquetes.

python -m pip install --upgrade pip

Actividad

Sigue los siguientes pasos y crea un documento con capturas de los comandos y el resultado de su ejecución (una captura por apartado):

1. Crea un entorno virtual (nombre a tu elección)
2. Activa el entorno
3. Lista los paquetes instalados
4. Instala el paquete pandas
5. Lista los paquetes instalados
6. Desactiva el entorno
7. Lista los paquetes instalados
8. Vuelve a activar el entorno
9. Desinstala el paquete pandas
10. Lista los paquetes instalados

Actividad

Instalar el entorno Jupyter en un nuevo entorno virtual:

```
pip install notebook
```

Y para arrancarlo:

```
jupyter notebook
```

Ahora podemos crear fácilmente ficheros de Python con el entorno ligero de Jupyter Notebook. Para ejecutar un fichero debemos abrir una consola y ejecutar el comando:

```
python nombre-fichero.py
```

Actividad

Crear con Jupyter Notebook un fichero de Python que pida al usuario su nombre y apellidos, los guarde en dos variables separadas y los muestre por pantalla. Ejecutar ese fichero por línea de comandos.